

National University of Computer and Emerging Sciences

Department of Computer Science



Chrono Rift

Multi-Process Tactical Combat Game

CS 2006 Operating Systems · Course Project

Semester 4 · Spring 2026

Submitted To

Dr. Faisal Cheema

Submitted By

Muhammad Ahmad Abbas

I24-3058

Hassan Amir

I24-0556

FAST – NATIONAL UNIVERSITY · ISLAMABAD CAMPUS

Contents

1	Introduction	3
1.1	Roll Number Seed	3
2	Game Architecture and Process Isolation	4
2.1	IPC Design	4
3	Synchronization and Concurrency Control	5
3.1	Lamport’s Bakery Algorithm (3-Process)	5
3.2	Dual Lock Architecture	5
4	Multithreading Model	6
4.1	Thread-per-NPC (ASP)	6
4.2	Thread-per-Player (HIP)	6
5	Temporal Scheduling and Stamina Logic	7
5.1	Stamina Accumulation	7
5.2	Arrival Time and Serial Execution	7
6	Signals and Asynchronous Interrupts	8
6.1	Stun Mechanic	8
6.2	Ultimate Ability	8
7	Weapons and Inventory Management	9
7.1	Inventory Allocator	9
7.2	Swap In/Out and Long-Term Storage	9
8	Deadlock Handling and Resource Management	10
8.1	Global Resource Table	10
8.2	Deadlock Detection	10
8.3	Eclipse Relic	10
9	Graphical Interface and Rendering	11
9.1	Visual Features	11
9.2	Gameplay Screenshots	11
9.3	Non-Blocking Design	14
10	Game Completion Conditions	15
11	Turnaround Time Analysis	16

11.1 Theoretical Model	16
11.2 Execution Overhead	16
12 Bonus: Local Multiplayer Extension	17
12.1 Implementation	17
12.2 Visual Differentiation	17
12.3 Competitive Scoring and Win Conditions	18
12.4 Synchronization	18
13 Build and Execution	19
13.1 Docker Environment	19
13.2 Build	19
13.3 Single Player	19
13.4 Multiplayer	19
14 Conclusion	20

1 Introduction

Chrono Rift is a multi-process tactical turn-based combat game built entirely in C++ using Linux system calls for inter-process communication and SFML for graphical rendering. The project demonstrates core operating system concepts including process isolation, shared memory IPC, multi-threaded execution, signal-based asynchronous interrupts, memory management with contiguous allocation, and deadlock detection.

The game pits a party of 1–4 human-controlled heroes against 2–9 computer-controlled enemies. A temporal scheduler based on stamina accumulation determines execution order, and all communication occurs through System V shared memory protected by Lamport’s Bakery Algorithm.

1.1 Roll Number Seed

All randomized elements use roll number seed **243058** (derived from I24-3058). The seed influences:

- **Player HP:** Roll number + random(100, 1000)
- **Enemy HP:** Last 2 digits (58) + random(50, 200)
- **Player Damage:** Last digit (8) + 10 = 18
- **Enemy Damage:** Second-last digit (5) + 10 = 15
- **Player Speed:** 100 / (number of players)
- **Enemy Speed:** random(10, 30)

2 Game Architecture and Process Isolation

The system is divided into three independent processes, each compiled into its own binary:

Process	Binary	Responsibility
Game Arbiter	<code>arbiter_bin</code>	Central authority: owns shared memory, enforces game rules, runs the temporal scheduler, manages the SFML rendering thread
Human Interfacing Process (HIP)	<code>hip_bin</code>	Captures keyboard input from the shared keyboard buffer, spawns one pthread per controlled hero, forwards actions to shared memory
Automated Strategic Process (ASP)	<code>asp_bin</code>	NPC decision-making engine; spawns one pthread per enemy, selects actions (Strike/Skip) based on game state

2.1 IPC Design

All inter-process communication uses **System V Shared Memory** (`shmget/shmat`). No pipes (named or unnamed) are used anywhere in the codebase.

The shared memory segment contains the complete `GameState` struct: entity arrays, action buffers (`hip_buf`, `asp_buf`), the Bakery lock, resource table, keyboard buffer, action log, and drop state.

Design Rule: Only the Arbiter writes game state fields. HIP writes only to `hip_buf`. ASP writes only to `asp_buf`. This design eliminates write conflicts by construction.

3 Synchronization and Concurrency Control

3.1 Lamport’s Bakery Algorithm (3-Process)

Synchronization is implemented using **Lamport’s Bakery Algorithm** configured for 3 participants (Arbiter, HIP, ASP). This is a memory-based mutual exclusion primitive using only plain `int` variables in shared memory.

The lock struct contains:

```
typedef struct {
    int choosing[3];
    int number[3];
} BakeryLock3;
```

Each process acquires the lock via `bakery3_enter()` before reading or writing shared state, and releases it via `bakery3_exit()`. Compiler memory barriers (`__asm__ (" ::: “memory”)`) prevent the optimizer from reordering memory operations across lock boundaries.

3.2 Dual Lock Architecture

Two independent Bakery locks protect different data:

- **lock** – protects the main game state (entities, buffers, phase)
- **res_lock** – protects the artifact resource table (Solar Core, Lunar Blade, Eclipse Relic ownership)

This separation prevents unnecessary contention when the Arbiter checks artifact ownership independently of normal game state access.

4 Multithreading Model

4.1 Thread-per-NPC (ASP)

The Automated Strategic Process creates one `pthread` per enemy entity. Each thread independently monitors whether it is the active entity (via `active_index` and `active_is_player`), selects an action when its turn arrives, and writes its decision to `asp_buf`.

4.2 Thread-per-Player (HIP)

The Human Interfacing Process creates one `pthread` per controlled hero. The main thread drains the shared keyboard buffer into a local input queue. Only the thread corresponding to the currently active player processes input; other player threads remain idle.

5 Temporal Scheduling and Stamina Logic

The game uses a stamina-based temporal scheduler running in the Arbiter's main loop.

5.1 Stamina Accumulation

Every scheduler tick (100ms), each alive entity's stamina increases by its speed value. When an entity's stamina reaches its maximum (100 for players, 150 for enemies), it becomes eligible to act.

5.2 Arrival Time and Serial Execution

The arrival time for each entity is computed as:

$$T_{\text{arrival}} = \frac{\text{max_stamina} - \text{current_stamina}}{\text{speed}} \quad (1)$$

The entity with the smallest arrival time acts first. **Execution is strictly serial:** the scheduler sets `active_index` to exactly one entity at a time, sets either `hip_go` or `asp_go`, and waits for the corresponding buffer to be filled before advancing.

After an action is committed, the acting entity's stamina is depleted to zero (or 50% for Skip), and the scheduler recalculates the next execution window.

6 Signals and Asynchronous Interrupts

6.1 Stun Mechanic

High-tier attacks can stun a target. The Arbiter delivers `SIGUSR1` to the target process via `kill(pid, SIGUSR1)`. The target's signal handler (registered via `sigaction()`) calls `sleep(3)` to halt execution for exactly 3 seconds. This is fully asynchronous – no flag polling is required.

6.2 Ultimate Ability

When a player holds both the Solar Core and Lunar Blade, they can trigger the Ultimate Ability. The Arbiter:

1. Sends `SIGSTOP` to the ASP process, freezing all enemy threads
2. Sets a 10-second timer using `alarm(10)`
3. When `SIGALRM` fires, the handler sends `SIGCONT` to resume the ASP

This implements signal-only suspension as required by the specification – no flags or pipes coordinate this state transition.

7 Weapons and Inventory Management

7.1 Inventory Allocator

Each player has a 20-slot linear inventory. Weapons require contiguous slots:

Weapon	Slots	Damage
Solar Core	10	95
Lunar Blade	10	90
Iron Halberd	7	55
Thunderstaff	6	50
Frostbow	6	48
Obsidian Axe	5	45
Venom Dagger	4	30
Splinter Stick	2	12

The allocator uses **First-Fit Contiguous Allocation**:

1. `inventory_find_first_fit()` scans for the first free region large enough
2. If no space exists, `inventory_list_runs()` identifies all weapon runs
3. `inventory_evict_run()` removes the minimum number of weapons to long-term storage
4. The weapon is placed in the freed contiguous space

7.2 Swap In/Out and Long-Term Storage

Players can swap weapons to/from long-term storage. Swapping in costs a full turn and the weapon cannot be used until the next turn (enforced via the `just_swapped_in` flag).

8 Deadlock Handling and Resource Management

8.1 Global Resource Table

A shared `ResourceTable` tracks ownership and waiters for three global artifacts: Solar Core, Lunar Blade, and Eclipse Relic. Access to this table is protected by a separate Bakery lock (`res_lock`).

8.2 Deadlock Detection

The Arbiter runs `deadlock_check_and_resolve_unsafe()` every scheduler tick. It detects circular wait: if entity A holds Solar Core and waits for Lunar Blade, while entity B holds Lunar Blade and waits for Solar Core, a deadlock is identified.

Resolution: The Arbiter forces the Lunar Blade holder to release it, breaking the cycle and allowing the game to proceed.

8.3 Eclipse Relic

The Eclipse Relic is introduced dynamically at runtime. When a weapon drop occurs and the relic is not yet present, there is a 20% chance it appears. Once introduced, it joins the global artifact pool with the same exclusivity and locking rules.

9 Graphical Interface and Rendering

The UI is implemented using SFML in a **dedicated rendering thread** (`std::thread`) that reads state from shared memory independently of the scheduler.

9.1 Visual Features

- Real-time HP and stamina bars for all entities
- Animated character sprites (6-frame sheets: 3 idle + 3 attack, 32x32 pixels)
- Per-hero textures (`Hero_1.png` through `Hero_4.png`)
- Two alternating NPC textures (`npc_1.png`, `npc_2.png`)
- Weapon arsenal grid with yellow highlight on active weapon
- Scrolling action log
- Floating damage numbers (red for player damage, yellow for enemy)
- Victory/Game Over overlay screens with match statistics
- Pause overlay (ESC) and controls help panel (F1)

9.2 Gameplay Screenshots

The following screenshots demonstrate the game's visual presentation during actual gameplay.

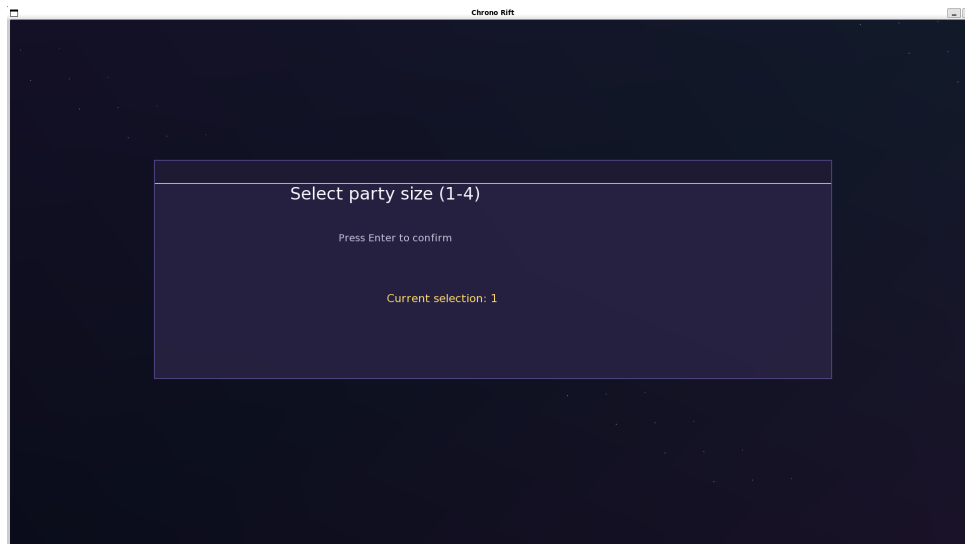


Figure 1. Party selection screen where the player chooses 1–4 heroes before battle.

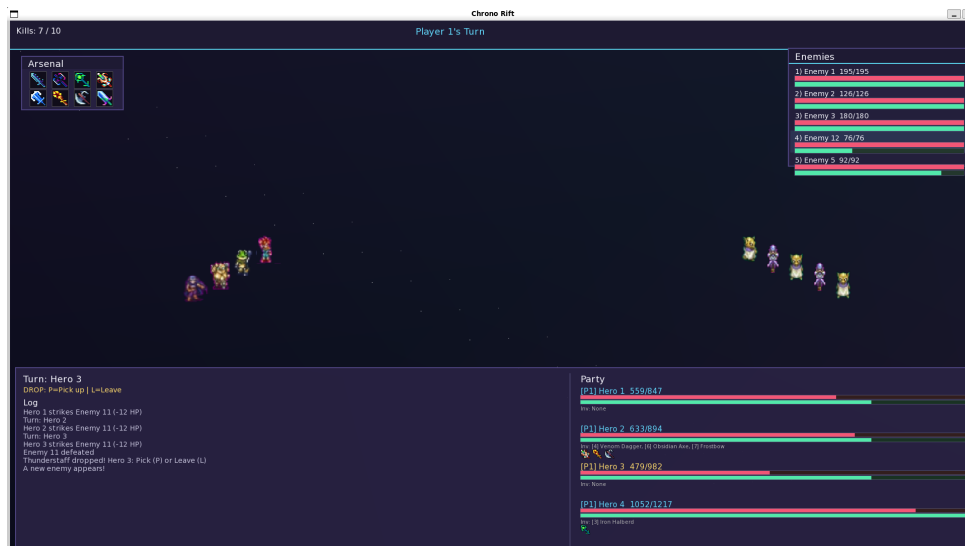


Figure 2. Main battle screen showing hero and enemy sprites, HP/stamina bars, weapon arsenal grid, action log, and party status panel.

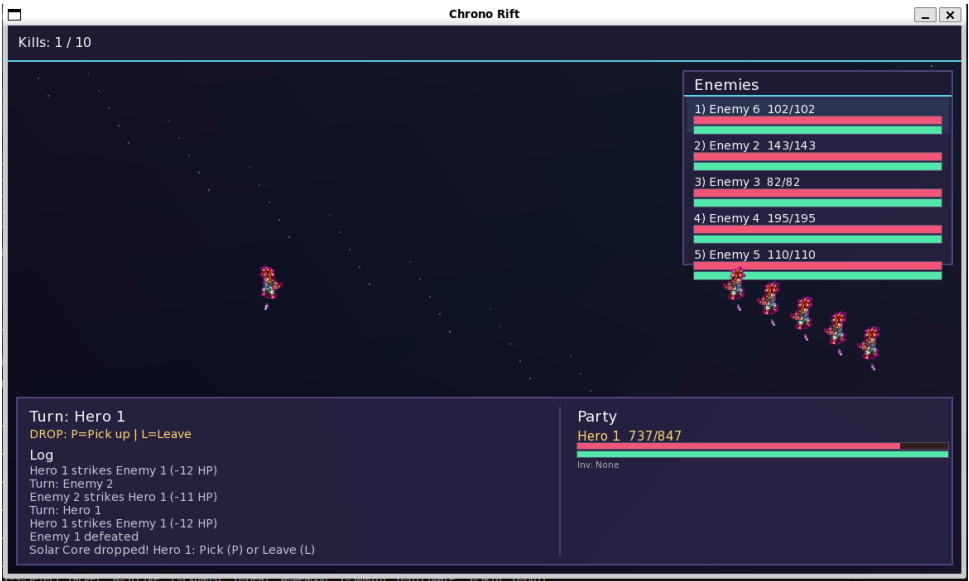


Figure 3. Weapon drop prompt – the player is offered a weapon after defeating an enemy.

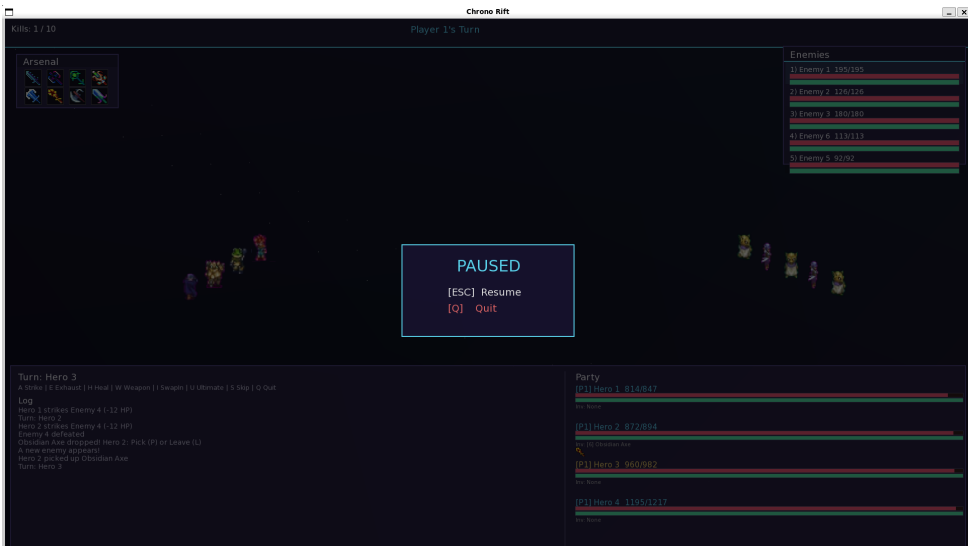


Figure 4. Pause overlay triggered by pressing ESC during battle.

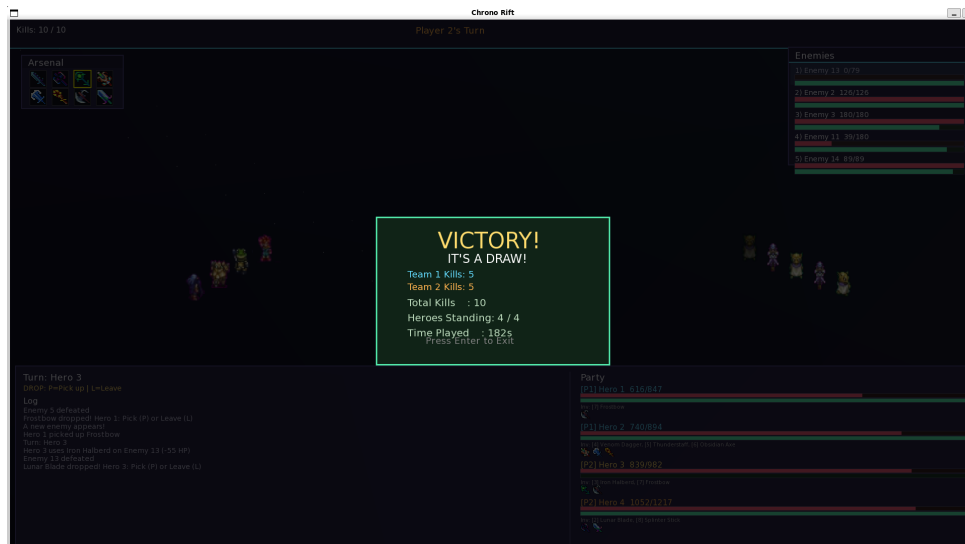


Figure 5. Victory screen displayed after defeating 10 enemies, showing match statistics.

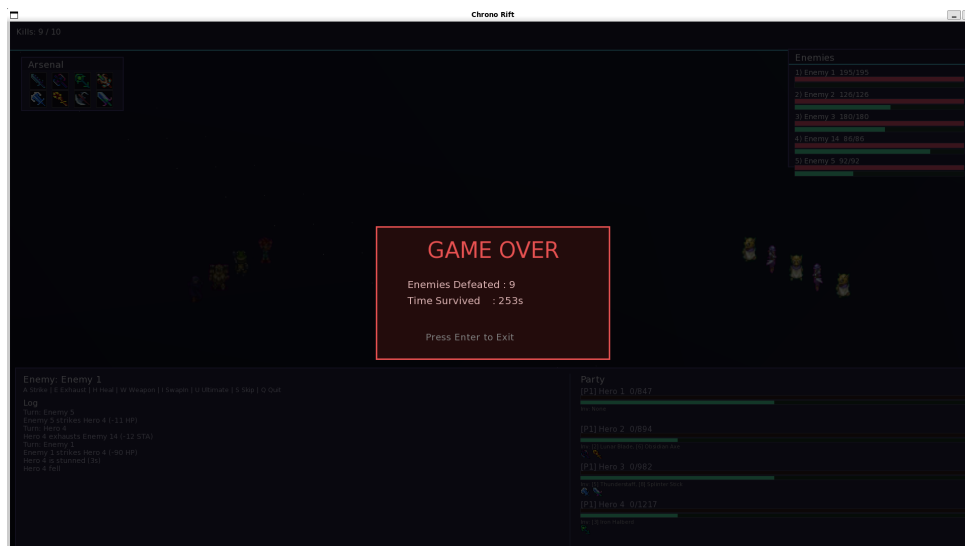


Figure 6. Game Over screen displayed when all player characters are defeated.

9.3 Non-Blocking Design

The render thread snapshots the entire game state under the Bakery lock once per frame (60 FPS) and draws from the local copy. This ensures the scheduler is never blocked by rendering.

10 Game Completion Conditions

The game exits gracefully under three conditions:

1. **Win:** Players kill a total of 10 enemies. A Victory screen displays match statistics and waits for Enter.
2. **Lose:** All player characters die. A Game Over screen is shown.
3. **Quit:** The player presses Q. HIP sends `SIGTERM` to the Arbiter, which cleans up shared memory and exits.

11 Turnaround Time Analysis

Turnaround time in Chrono Rift measures the duration from when an entity's stamina begins filling to when its action completes execution.

11.1 Theoretical Model

For an entity with speed s and maximum stamina $S_{\max}$:

$$T_{\text{fill}} = \frac{S_{\max}}{s} \times T_{\text{tick}} \quad (2)$$

where $T_{\text{tick}} = 100\text{ms}$. For a player with 4 heroes: speed = $100/4 = 25$, so:

$$T_{\text{fill}} = \frac{100}{25} \times 100\text{ms} = 400\text{ms}$$

For enemies with speed 20 and max stamina 150:

$$T_{\text{fill}} = \frac{150}{20} \times 100\text{ms} = 750\text{ms}$$

11.2 Execution Overhead

The total turnaround includes:

- **Stamina fill time:** Computed above
- **Input latency:** Time for HIP/ASP to respond (<50ms typically)
- **Lock contention:** Bakery lock acquisition (<1ms under normal load)
- **Action processing:** Arbiter applies damage/healing (<1ms)

The serial execution model ensures predictable turnaround since only one entity acts at a time, eliminating scheduling conflicts.

12 Bonus: Local Multiplayer Extension

The project includes an optional local multiplayer mode where two separate HIP processes control different subsets of heroes.

12.1 Implementation

Each HIP process accepts command-line arguments specifying which player indices it controls:

```
./hip_bin 0 1 &      # Player 1 controls heroes 0 and 1
./hip_bin 2 3 &      # Player 2 controls heroes 2 and 3
```

Each HIP writes its team assignment into a `team[4]` array in shared memory. The Arbiter detects multiplayer by checking if any team assignments exist, and automatically starts the game with 4 heroes.

12.2 Visual Differentiation

In multiplayer mode, the UI provides:

- Hero names prefixed with **[P1]** or **[P2]**
- Team-colored names: Team 1 in cyan, Team 2 in orange
- A “Player X’s Turn” banner in the top bar during battle

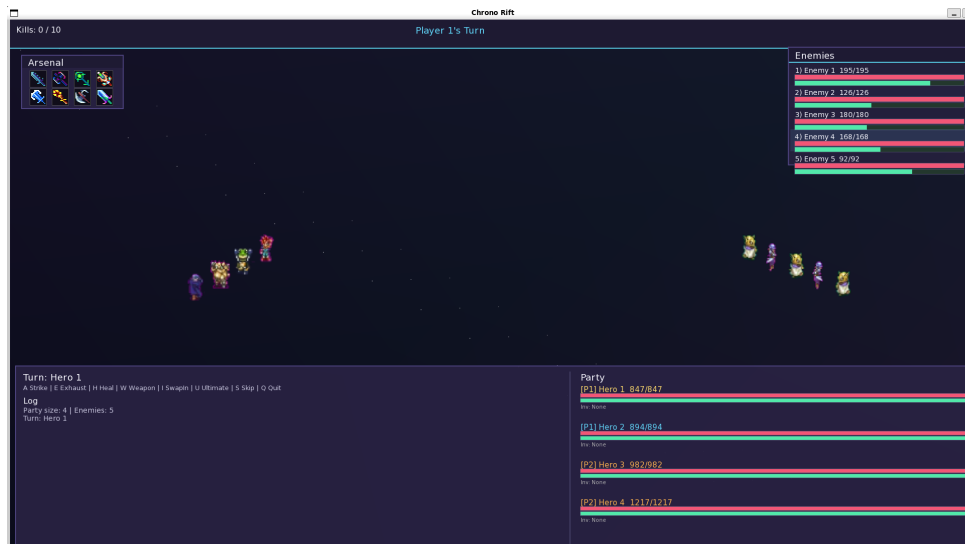


Figure 7. Multiplayer mode showing team-colored hero names ($[P1]/[P2]$ labels) and the turn indicator banner.

12.3 Competitive Scoring and Win Conditions

In Multiplayer mode, the game tracks kills independently for each team using a `team_kills[3]` array in shared memory. When an enemy is defeated, the Arbiter credits the kill to the team of the currently active player.

The end-game results are determined as follows:

- **Victory:** If the players reach the global kill target, the team with the higher number of kills is declared the winner (e.g., “TEAM 1 WINS!”). If kills are equal, a draw is announced.
- **Total Defeat:** If all player characters from both teams are killed, the game enters a special Game Over state indicating “BOTH TEAMS DEFEATED”, while still displaying final scores to show relative performance.

12.4 Synchronization

Both HIP processes share the same Bakery-locked shared memory. Each only pops keys from the shared keyboard buffer when it is currently the turn of a hero it controls, preventing input conflicts.

13 Build and Execution

13.1 Docker Environment

The project runs inside a Docker container based on `ubuntu:22.04` with `libsFML-dev` installed. The `Dockerfile` and `requirements.txt` are included.

13.2 Build

```
make          # Builds arbiter_bin, hip_bin, asp_bin
```

13.3 Single Player

```
./asp_bin &  
./hip_bin 0 1 2 3 &  
./arbiter_bin
```

13.4 Multiplayer

```
./asp_bin &  
./hip_bin 0 1 &  
./hip_bin 2 3 &  
./arbiter_bin
```

14 Conclusion

Chrono Rift successfully demonstrates the practical application of core operating system concepts in an interactive, real-time game environment. The project implements process isolation with three independent binaries, shared memory IPC without pipes, Lamport's Bakery Algorithm for synchronization, thread-per-entity concurrency, stamina-based temporal scheduling, signal-driven asynchronous mechanics, first-fit contiguous memory allocation, and circular-wait deadlock detection with automated resolution. The optional multiplayer extension further demonstrates the robustness of the shared memory architecture under increased process concurrency.

References

- [1] A. S. Tanenbaum and H. Bos, “Modern Operating Systems,” *Pearson*, 4th Edition, 2015.
- [2] L. Lamport, “A New Solution of Dijkstra’s Concurrent Programming Problem,” *Communications of the ACM*, vol. 17, no. 8, pp. 453–455, 1974.
- [3] SFML Development Team, “Simple and Fast Multimedia Library – Documentation,” 2024. <https://www.sfm1-dev.org/documentation/>